DOCUMENTACIÓN TÉCNICA

Fase II – Analizador Léxico y Tabla de Símbolos

1. Descripción General de la Fase

En esta fase del proyecto se desarrollaron los módulos correspondientes al **Análisis Léxico** y la **Tabla de Símbolos**, implementados en Python y organizados en archivos independientes para mantener una arquitectura modular y escalable.

Los componentes desarrollados fueron:

- `lexer.py`
- `tabla_simbolos.py`
- `test_lexer.py`
- `test_tabla_simbolos.py`
- `requirements.txt`

Esta organización permite separar responsabilidades, facilitar pruebas unitarias y mantener un control estructurado del sistema.

2. Arquitectura del Módulo

La estructura implementada sigue un enfoque modular dividido en:

2.1 Módulo de Análisis Léxico (`lexer.py`)

Encargado de:

- Leer el código fuente
- Identificar lexemas
- Clasificarlos como tokens
- Detectar errores léxicos
- Generar la secuencia de tokens

Este módulo representa la primera fase del compilador y se fundamenta en el reconocimiento de patrones mediante expresiones regulares, alineado con la teoría de lenguajes regulares.

2.2 Módulo de Tabla de Símbolos (`tabla_simbolos.py`)

Encargado de:

- Almacenar identificadores encontrados
- Evitar duplicados

- Registrar atributos asociados
- Servir de estructura base para fases posteriores

Este módulo permite centralizar la información semántica preliminar del programa analizado.

2.3 Módulos de Prueba

Se desarrollaron pruebas unitarias independientes:

- `test_lexer.py`
- `test_tabla_simbolos.py`

Estas pruebas permiten validar:

- Reconocimiento correcto de tokens
- Inserción adecuada en la tabla de símbolos
- Manejo de errores
- Comportamiento esperado ante entradas inválidas

La inclusión de pruebas mejora la confiabilidad y mantenibilidad del sistema.

2.4 Archivo de Dependencias (`requirements.txt`)

Este archivo define las bibliotecas necesarias para ejecutar el proyecto, garantizando que cualquier miembro del equipo pueda reproducir el entorno de ejecución correctamente.

3. Desarrollo del Analizador Léxico

El analizador léxico fue implementado en Python utilizando mecanismos de reconocimiento de patrones para identificar los distintos componentes léxicos del lenguaje.

3.1 Funcionamiento General

El proceso seguido por el lexer es:

1. Leer la entrada carácter por carácter.
2. Agrupar secuencias significativas (lexemas).
3. Comparar los lexemas con patrones definidos.
4. Generar el token correspondiente.
5. Registrar identificadores en la tabla de símbolos.
6. Reportar errores en caso de caracteres inválidos.

3.2 Clasificación de Tokens

El lexer reconoce diferentes categorías léxicas como:

- Identificadores
- Constantes numéricas
- Palabras reservadas
- Operadores
- Símbolos especiales

Cada token se representa mediante una estructura que contiene:

- Tipo de token
- Valor del lexema
- Línea (si fue implementado)
- Posición (si aplica)

3.3 Manejo de Errores Léxicos

Se implementó detección de errores cuando un carácter no coincide con ningún patrón válido.

El sistema:

- Identifica el error
- Muestra información contextual
- Continúa el análisis

Esto evita que el compilador se detenga ante el primer error, mejorando la experiencia de análisis.

4. Implementación de la Tabla de Símbolos

La tabla de símbolos fue diseñada como una estructura dinámica, permitiendo almacenar información relevante sobre cada identificador detectado.

4.1 Funcionalidades Principales

- Inserción de símbolos
- Verificación de existencia previa
- Consulta de información
- Actualización de atributos

4.2 Estructura de Datos

Internamente se utilizó una estructura tipo diccionario, lo que permite:

- Acceso rápido (O(1) promedio)

- Evitar duplicación
- Organización clara por clave (identificador)

5. Pruebas Unitarias

Para garantizar el correcto funcionamiento del sistema se desarrollaron pruebas independientes:

5.1 Pruebas del Lexer

Validan:

- Reconocimiento correcto de tokens
- Identificación de errores
- Manejo de entradas complejas

5.2 Pruebas de la Tabla de Símbolos

Validan:

- Inserción correcta
- No duplicación
- Consulta eficiente
- Comportamiento ante símbolos inexistentes

El uso de pruebas unitarias refuerza la calidad del desarrollo y demuestra aplicación de buenas prácticas de ingeniería de software.

6. Integración con el Repositorio

El desarrollo se realizó utilizando control de versiones mediante Git y GitHub, permitiendo:

- Trabajo colaborativo
- Manejo de ramas
- Control de cambios
- Integración de módulos

Cada archivo fue desarrollado en su respectiva rama antes de integrarse al repositorio principal.

7. Conclusiones Técnicas

El desarrollo del analizador léxico y la tabla de símbolos permitió aplicar conceptos fundamentales de:

- Lenguajes formales
- Autómatas finitos
- Expresiones regulares
- Estructuras de datos
- Pruebas unitarias
- Control de versiones

Esta fase constituye la base estructural del compilador, ya que provee la representación formal del código fuente necesaria para el análisis sintáctico y semántico posterior.

La modularidad implementada facilita la escalabilidad del proyecto y permite futuras ampliaciones sin afectar la arquitectura base.

Documentación del Analizador Sintáctico (parser.py)

1. Descripción General

El módulo reglas_parser.py implementa el analizador sintáctico del lenguaje utilizando la librería PLY (Python Lex-Yacc).

Su función es verificar que la secuencia de tokens generada por el analizador léxico cumpla la gramática definida del lenguaje.

El parser también:

Construye una estructura tipo árbol sintáctico (AST simplificado) usando tuplas.

Detecta errores sintácticos.

Permite continuar el análisis después de encontrar errores.

2. Importación de Librerías

```
import ply.yacc as yacc
from lexer import tokens, lista_errores, encontrar_columna
```

Descripción

Elemento	Función
yacc	Generador de analizadores sintácticos de PLY
tokens	Lista de tokens definidos en el lexer
lista_errores	Lista donde se almacenan los errores
encontrar_columna	Función para calcular la columna de un token

3. Precedencia de Operadores

```
precedence = (
    ('left', 'OR'),
    ('left', 'AND'),
    ('left', 'IGUAL', 'DIFERENTE'),
    ('left', 'MENOR', 'MAYOR', 'MENOR_IGUAL', 'MAYOR_IGUAL'),
    ('left', 'MAS', 'MENOS'),
    ('left', 'MULTIPLICACION', 'DIVIDIR', 'MODULO'),
    ('right', 'NOT', 'UMENOS')
)
```

Define la prioridad y asociatividad de operadores para evitar ambigüedades en expresiones.

Orden de precedencia (de menor a mayor)

Operador	Tipo
OR	lógico
AND	lógico
== , !=	comparación
< > <= >=	relacional
+ -	aritmético
* / %	aritmético
NOT	lógico
- (unario)	negación

Ejemplo interpretado correctamente:

a + b * c

Se evalúa como:

a + (b * c)

4. <u>Regla Inicial del Programa</u>

```
def p_programa(p):
    '''programa : PROGRAMA LLAVE_IZQ sentencias LLAVE_DER'''
```

Define la estructura principal del programa.

Forma del programa

```
PROGRAMA {
    sentencias
}
```

AST generado

('programa', sentencias)

5. Lista de Sentencias

```
def p_sentencias(p):
    '''sentencias : sentencias sentencia | sentencia'''
```

Permite múltiples instrucciones dentro del programa.

Ejemplo:

```
ENTERO x;
x = 5;
IMPRIMIR(x);
```

Se guarda como una lista de sentencias.

6. Tipos de Sentencias

```
def p_sentencia(p):
```

Tipos soportados:

Tipo	Descripción
declaracion	declaración de variables
asignacion	asignación de valores
sentencia_si	estructura condicional
sentencia_mientras	ciclo while
sentencia_hacer_mientras	ciclo do-while
sentencia_para	ciclo for
sentencia_imprimir	salida de datos
sentencia_leer	entrada de datos

7. Tipos de Datos

```
def p_tipo(p):
    '''tipo : ENTERO | DECIMAL | CADENA_TIPO | BOOLEANO'''
```

Tipos soportados:

Tipo	Ejemplo
ENTERO	10
DECIMAL	3.14
CADENA_TIPO	"hola"
BOOLEANO	verdadero/falso

8. Declaración de Variables

def p_declaracion(p):

Permite dos formas:

Declaración simple
ENTERO x;

AST:

('declaracion', 'ENTERO', 'x', None)
Declaración con valor
ENTERO x = 10;

AST:

('declaracion', 'ENTERO', 'x', 10)

9. Asignación de Variables

def p_asignacion(p):

Ejemplo:

x = 20;

AST:

('asignacion', 'x', 20)

10. Condicional if

def p_sentencia_si(p):
Forma simple
SI (condicion) {
    sentencias
}
Forma con else
SI (condicion) {
    sentencias
}
SINO {
    sentencias
}

AST:

('si', condicion, bloque_si, bloque_sino)

11. Ciclo while

def p_sentencia_mientras(p):

Ejemplo:

MIENTRAS (x < 10) {
    x = x + 1;
}

AST:

('mientras', condicion, sentencias)

12. <u>Ciclo do while</u>

def p_sentencia_hacer_mientras(p):

Ejemplo:

```
HACER_MIENTRAS {
   x = x + 1;
} MIENTRAS (x < 10);
```

AST:

('hacer_mientras', sentencias, condicion)

13. <u>Ciclo for</u>

def p_sentencia_para(p):

Estructura:

```
PARA (inicializacion; condicion; incremento) {
    sentencias
}
```

Ejemplo:

```
PARA (i=0; i<10; i=i+1) {
   imprimir(i);
}
```

AST:

('para', inicializacion, condicion, incremento, sentencias)

14. Instrucción imprimir

def p_sentencia_imprimir(p):

Ejemplo:

IMPRIMIR(x);

AST:

('imprimir', x)

15. Instrucción leer

def p_sentencia_leer(p):

Ejemplo:

LEER(x);

AST:

('leer', x)

16. Expresiones

Las expresiones pueden incluir:

Operadores aritméticos
+
-
*
/
%
Operadores relacionales
<
>

<=
>=
==
!=
Operadores lógicos
AND
OR
NOT

Ejemplo:

x + y

AST:

('+', x, y)

17. <u>Expresiones Unarias</u>

Negación lógica
NOT expresion

AST:

('not', expresion)
Negación numérica
-5

AST:
('negativo', 5)

18. Literales

def p_expresion_literal(p):

Valores permitidos:

Tipo	Ejemplo
entero	10
decimal	3.5
cadena	"hola"
booleano	verdadero
identificador	x

19. Manejo de Errores Sintácticos

def p_error(p):

Cuando el parser detecta un error:

Se guarda en lista_errores.

Ejemplo:

token inesperado: '}'

Se registra:

Campo	Valor
tipo	Sintactico
descripcion	token inesperado
linea	línea del error
columna	columna del error

Recuperación del Error

El parser intenta recuperarse ignorando tokens hasta encontrar:

;

Esto permite continuar analizando el resto del programa.

20. Construcción del Parser
parser = yacc.yacc()

Esta línea genera automáticamente el analizador sintáctico LR usando todas las reglas definidas.

21. Flujo del Análisis Sintáctico

Proceso completo:

Código fuente
Análisis léxico (tokens)
Parser (reglas gramaticales)
Verificación sintáctica
Generación de AST
Reporte de errores

Documentación del Módulo reportes.py

1. Descripción General

El módulo reportes.py se encarga de generar reportes en formato HTML a partir de los resultados obtenidos durante el análisis del programa.

Estos reportes permiten visualizar de manera clara:

Los tokens reconocidos por el analizador léxico

Los errores detectados durante el análisis

La tabla de símbolos generada por el compilador

Los reportes se guardan como archivos HTML, lo que permite abrirlos fácilmente en cualquier navegador web.

2. Funciones del Módulo

El módulo contiene tres funciones principales:

Función	Descripción
generar_html_tokens	Genera el reporte de tokens
generar_html_errores	Genera el reporte de errores
generar_html_tabla_simbolos	Genera el reporte de la tabla de símbolos

3. Función generar_html_tokens

Definición

```
def generar_html_tokens(listaTokens, rutaSalida):
```

Descripción

Genera un reporte HTML que muestra todos los tokens encontrados durante el análisis léxico.

Parámetros

Parámetro	Descripción
listaTokens	Lista de tokens generados por el lexer

rutaSalida Ruta donde se guardará el archivo HTML

Estructura del Reporte

El reporte contiene una tabla con la siguiente información:

Campo	Descripción
#	Número del token
Lexema	Valor del token
Tipo de Token	Tipo reconocido
Línea	Línea donde aparece
Columna	Columna donde aparece

Ejemplo de tabla generada

#	Lexema	Tipo	Línea	Columna
1	PROGRAMA	PALABRA_RESERVADA	1	1
2	x	IDENTIFICADOR	2	8
3	10	NUMERO_ENTERO	2	12

Funcionamiento interno

El programa recorre la lista de tokens utilizando:

enumerate(listaTokens, 1)

Esto permite numerar automáticamente cada token.

Luego se construyen las filas de la tabla HTML.

Finalmente el archivo se guarda usando:

with open(rutaSalida, 'w', encoding='utf-8')

4. Función generar_html_errores
Definición
def generar_html_errores(listaErrores, rutaSalida):
Descripción

Genera un reporte HTML que muestra los errores detectados durante el análisis léxico y sintáctico.

Parámetros

Parámetro	Descripción
listaErrores	Lista de errores detectados
rutaSalida	Archivo HTML donde se guardará el reporte

Información mostrada
La tabla del reporte incluye:

Campo	Descripción
#	Número del error
Tipo de Error	Léxico o sintáctico
Descripción	Explicación del error
Línea	Línea donde ocurre
Columna	Columna del error

Ejemplo de salida

#	Tipo	Descripción	Línea	Columna
1	Léxico	carácter no reconocido	4	10
2	Sintáctico	token inesperado	6	3

Conteo de errores

El reporte también calcula el total de errores usando:

```
sum(1 for e in listaErrores if e['tipo'] == 'Lexico')

sum(1 for e in listaErrores if e['tipo'] == 'Sintactico')
```

Esto permite mostrar un resumen como:

Total errores léxicos: 2
Total errores sintácticos: 1

5. Función generar_html_tabla_simbolos

Definición

def generar_html_tabla_simbolos(tabla, rutaSalida):

Descripción

Genera un reporte HTML que muestra la tabla de símbolos del programa.

Parámetros

Parámetro	Descripción
tabla	Objeto de tipo TablaSimbolos
rutaSalida	Archivo HTML de salida

Información mostrada

La tabla contiene:

Campo	Descripción
#	Número del símbolo
Nombre	Identificador
Tipo de Dato	Tipo de variable
Valor	Valor almacenado
Línea	Línea de declaración
Columna	Columna de declaración

Ejemplo de reporte

#	Nombre	Tipo	Valor	Línea	Columna
1	x	ENTERO	10	2	5
2	nombre	CADENA	"Ana"	4	8

Manejo de valores nulos

Si el símbolo no tiene valor asignado, se muestra:

-

Esto se logra con:

```
valor = sim.valor if sim.valor is not None else '-'
```

6. Flujo de Generación de Reportes

El proceso de generación de reportes funciona de la siguiente forma:

Código fuente
Análisis léxico
Lista de tokens
generar_html_tokens()
ReporteTokens.html
Errores detectados
generar_html_errores()
ReporteErrores.html
Tabla de símbolos
generar_html_tabla_simbolos()
ReporteSimbolos.html

Documentación del Módulo main.py

1. Descripción General

El archivo main.py es el punto de entrada del sistema de análisis del lenguaje.
Su función principal es ejecutar las fases iniciales de un compilador:

Análisis Léxico

Análisis Sintáctico

Construcción de la Tabla de Símbolos

Reporte de resultados y errores

El programa recibe como parámetro un archivo de código fuente, lo procesa utilizando el lexer y el parser, y finalmente genera un resumen del análisis.

2. Importación de Módulos

```
import sys
import os
from lexer import lexer, tokens, lista_errores, encontrar_columna
from reglas_parser import parser
from tabla_simbolos import TablaSimbolos
```

Descripción

Módulo	Función
sys	Permite acceder a los argumentos enviados por línea de comandos
os	Manejo de rutas y sistema operativo
lexer	Contiene el analizador léxico del lenguaje
tokens	Lista de tokens definidos en el lexer
lista_errores	Lista donde se almacenan errores léxicos y sintácticos
encontrar_columna	Función que calcula la columna donde aparece un token
parser	Analizador sintáctico del lenguaje
TablaSimbolos	Clase para almacenar los identificadores detectados

3. Recepción de Parámetros

```
if len(sys.argv) < 2:
    print("Uso: python main.py <archivo>")
    sys.exit(1)
```

Funcionamiento

El programa requiere que el usuario indique el archivo a analizar.

Ejemplo de ejecución:

```
python main.py programa.txt
```

Si no se proporciona un archivo, el programa termina mostrando el mensaje de uso.

4. Lectura del Archivo Fuente

```
ruta = sys.argv[1]

try:
    with open(ruta, 'r', encoding='utf-8') as f
        contenido = f.read()
except FileNotFoundError:
    print(f"Error, no se encontró el siguiente archivo: {ruta}")
    sys.exit(1)
```

Descripción

Se obtiene la ruta del archivo desde los argumentos.

Se intenta abrir el archivo en modo lectura.

Si el archivo no existe, el programa muestra un error y termina.

El contenido leído se almacena en la variable: contenido

5. Fase de Análisis Léxico

```
print("Fase de análisis léxico:")
lista_errores.clear()
lexer.lineno = 1
lexer.input(contenido)
```

Objetivo

El análisis léxico divide el código fuente en tokens.

Antes de iniciar:

Se limpia la lista de errores.

Se reinicia el contador de líneas del lexer.

Obtención de Tokens

```
lista_tokens = []
while True:
    tok = lexer.token()
    if not tok:
        break
```

El lexer procesa el texto y va generando tokens uno por uno.

Cada token contiene:
Valor
Tipo
Línea
Columna

Almacenamiento de Tokens

```
lista_tokens.append({
    'valor': tok.value,
    'tipo': tok.type,
    'linea': tok.lineno,
    'columna': encontrar_columna(contenido, tok)
})
```

Cada token se guarda en una lista como un diccionario.

Ejemplo de token almacenado:

```
{
 valor: "x",
 tipo: "IDENTIFICADOR",
 linea: 3,
 columna: 5
}
```

Conteo de Tokens

```
print(f"Se encontraron {len(lista_tokens)} tokens")
```

Muestra el total de tokens generados.

6. <u>Fase de Análisis Sintáctico</u>

```
print("\nFase de análisis sintáctico:")
lexer.lineno = 1
resultado = parser.parse(contenido, lexer=lexer)
```

Objetivo

El parser analiza la estructura del programa para verificar si cumple la gramática definida.

Si existen errores de sintaxis, estos se almacenan en:

lista_errores

7. Clasificación de Errores

```
errores_lexicos = [e for e in lista_errores if e['tipo'] == 'Lexico']
errores_sintacticos = [e for e in lista_errores if e['tipo'] == 'Sintactico']
```

Los errores se dividen en:

Tipo	Descripción
Léxicos	Caracteres o símbolos inválidos
Sintácticos	Estructura incorrecta del programa

Luego se muestra el total:

Se encontraron X errores léxicos
Se encontraron X errores sintácticos

8. Construcción de la Tabla de Símbolos

```
tabla = TablaSimbolos()
```

La tabla de símbolos almacena las variables declaradas en el programa.

Tipos de Datos Permitidos

```
tipos_validos = {'ENTERO', 'DECIMAL', 'CADENA_TIPO', 'BOOLEANO'}
```

Estos representan los tipos de variables válidos del lenguaje.

Detección de Declaraciones

```
for i in range(len(lista_tokens) - 1):
    if lista_tokens[i]['tipo'] in tipos_validos and lista_tokens[i + 1]['tipo'] ==
'IDENTIFICADOR':
```

El programa detecta una declaración cuando encuentra:

TIPO IDENTIFICADOR

Ejemplo:

ENTERO x

DECIMAL precio
BOOLEANO activo
Inserción en la Tabla de Símbolos

```
tabla.agregar(
    nombre=lista_tokens[i + 1]['valor'],
    tipo=lista_tokens[i]['tipo'],
    linea=lista_tokens[i + 1]['linea'],
    columna=lista_tokens[i + 1]['columna']
)
```

Se guarda:

Campo	Descripción
nombre	Identificador
tipo	Tipo de dato
línea	Línea donde aparece
columna	Columna donde aparece

9. Reporte Final

```
print("\nResumen de analisis:")
print(f"Total de tokens: {len(lista_tokens)}")
print(f"Total de errores: {len(lista_errores)}")
```

El sistema muestra un resumen general del análisis realizado.

10. Impresión de Errores

```
for e in lista_errores:
    print(f"error: {e['tipo']}, línea: {e['linea']}, columna: {e['columna']}, mensaje:
{e['descripcion']}")
```

Cada error muestra:
Tipo de error
Línea
Columna
Descripción

Ejemplo:

error: Lexico, línea: 4, columna: 10, mensaje: carácter no reconocido

11. Impresión de la Tabla de Símbolos

```
print(f"Simbolos: {len(tabla.obtener_todos())}")
```

Luego se listan todos los símbolos:

```
for s in tabla.obtener_todos():
    print(f"simbolo: {s.nombre}, tipo: {s.tipo}")
```

Ejemplo de salida:

Simbolos: 3
simbolo: x, tipo: ENTERO
simbolo: precio, tipo: DECIMAL
simbolo: activo, tipo: BOOLEANO

12. Flujo General del Programa

El proceso completo del sistema sigue el siguiente flujo:

Inicio
Leer archivo fuente
Análisis Léxico
Generación de Tokens
Análisis Sintáctico
Detección de Errores

Construcción de Tabla de Símbolos
Mostrar Resultados
Fin

DOCUMENTACIÓN — PARTE 4 (P4)

Elaborado por P4

1. Módulo de Manejo de Errores (errores.py)

Módulo centralizado que almacena y clasifica los errores detectados durante el análisis léxico y sintáctico.

Funciones disponibles

Función	Descripción
agregar_lexico(msg, linea, valor)	Registra un error léxico
agregar_sintactico(msg, linea, valor)	Registra un error sintáctico
todos()	Retorna todos los errores ordenados
generar_html()	Genera el HTML de la bitácora
para_reportes()	Convierte errores al formato de reportes.py

Variables internas

Variable	Descripción
errores_lexicos	Lista de errores léxicos
errores_sintacticos	Lista de errores sintácticos

Uso desde otros módulos

```
import errores
errores.agregar_lexico("Caracter ilegal '@'", linea=2, valor="@")
errores.agregar_sintactico("Token inesperado", linea=3, valor="entero")
html = errores.generar_html()
```

2. Pruebas Unitarias (test_errores.py)

Archivo de pruebas que verifica el correcto funcionamiento del módulo errores.py de forma aislada.

Ejecutar con: python test_errores.py

Pruebas incluidas: 1. Agregar error léxico correctamente 2. Agregar error sintáctico correctamente 3. todos() devuelve ambos tipos de errores 4. Ordenamiento por línea correcto 5. HTML generado correctamente 6. HTML sin errores muestra mensaje correcto

3. Archivos de Prueba (carpeta pruebas/)

prueba_valida_01.txt

Propósito: verificar que el compilador acepta un programa correcto con declaraciones, asignación y condicional.

```
programa {
    entero x = 10;
    entero y = 5;
    entero resultado = 0;
    resultado = x + y;
    si (resultado > 10) {
        imprimir(resultado);
    }
}
```

Resultado esperado: 0 errores léxicos, 0 errores sintácticos.

prueba_valida_02.txt

Propósito: verificar que el compilador acepta un programa correcto con condicional, ciclo mientras y ciclo para.

```
programa {
    entero a = 3;
    entero b = 7;
    decimal promedio = 8.75;
    entero contador = 1;

    si (a < b) {
        imprimir(a);
    }

    mientras (contador < 6) {
        imprimir(contador);
        contador = contador + 1;
    }

    para (contador = 1; contador < 4; contador = contador + 1) {
```

```
        imprimir(contador);
    }
}
```

Resultado esperado: 0 errores léxicos, 0 errores sintácticos.

prueba_errores_lexicos.txt

Propósito: verificar que el lexer detecta caracteres no reconocidos por el lenguaje.

```
programa {
    entero x = 10;
    entero y @ 5;
    entero z = 3;
}
```

Error intencional: '@' en línea 3 es un carácter ilegal. Resultado esperado: 1 error léxico.

prueba_errores_sintacticos.txt

Propósito: verificar que el parser detecta violaciones a las reglas gramaticales del lenguaje.

```
programa {
    entero x = 10
    entero y = 5;
}
```

Error intencional: falta ';' al final de la línea 2. Resultado esperado: 1 error sintáctico.

prueba_mixta.txt

Propósito: verificar que el compilador detecta ambos tipos de errores en un mismo programa y continúa el análisis.

```
programa {
    entero x = 10;
    entero y @ 5;
    entero z = 3
}
```

Errores intencionales: - '@' en línea 3 (error léxico) - falta ';' en línea 4 (error sintáctico) Resultado esperado: 1 error léxico, 1 error sintáctico.